Edge cases (a request that fails validation) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The reason this works without you writing anything new: FastAPI checks the request body against `ChatRequest` *before* your route function's body ever runs — so if `message` is missing, your `INSERT` is never reached. That's automatic, not something you coded yourself.

The harder question: **could a request that *passes* validation still leave a partial write, if your route does two database writes and fails between them?** For example: insert a "started" row, run the logic, then update the row to "completed". If the logic crashes in the middle, the "started" row stays behind forever — a silent partial write that 422 handling can't prevent, because the request was valid all along.

The exact pieces:

- `requests.post(url, json={})` (from Doc02's `requests` library) is easier to script than `curl`: `response = requests.post("http://localhost:8000/chat", json={})`, then `assert response.status_code == 422`.
- `response.json()` on a 422 shows FastAPI's error detail — exactly which field failed and why, e.g. `{"detail": [{"loc": ["body", "message"], "msg": "Field required", ...}]}`.
- Count rows before and after the bad request, and `assert` they're equal — don't just eyeball it once.
- `TestClient(app)` from `health_route` — the same check without a running server.
- `cursor = conn.execute("INSERT ...")` then `cursor.lastrowid` — the `id` of the row you just inserted, so a later `UPDATE ... WHERE id = ?` can find it.
- `with conn:` around both writes (the README's transaction pattern) — if anything inside raises, *both* writes are undone, so no half-finished row is left.

Hint 2 — The plan, and almost the whole thing

```
before_count = SELECT COUNT(*) FROM runs
response = POST /chat with {}   # missing "message"
assert response.status_code == 422
assert before_count == SELECT COUNT(*) FROM runs

then, for a route with TWO writes per request:
    with conn:                       # one transaction
        insert a row with status "started", keep cursor.lastrowid
        run the logic (a fake failure on a magic message)
        update that row: output, status "completed"
    send the magic message, confirm 500 and the row count didn't change
```

Here's most of the two-write route — fill in the transaction yourself:

```
# practice/sqlite_persistence_practice.py — Edge cases section
import sqlite3
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("two_step.db", check_same_thread=False)
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        status TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

def run_chat_two_step(message: str) -> str:
    if message == "fail-after-insert":
        raise RuntimeError("simulated failure between steps")
    return "You said: " + message

@app.post("/chat")
def chat(request: ChatRequest):
    try:
        # your turn: inside "with conn:", INSERT a "started" row,
        # call run_chat_two_step(), then UPDATE the row to "completed"
```

```
        ...
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    return {"reply": reply}
```

Full solution: Show me the solution